

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

2026.06.26

Spring Boot 시작 구조와 구동 원리

핵심

`D:/Study/Spring/SpringSample`은 Spring Boot로 만든 첫 샘플 프로젝트다. 지금 단계에서는 DB 연결보다 "어디서 실행이 시작되고, 요청이 어떤 과정을 거쳐 Controller와 HTML까지 가는지"를 이해하는 것이 중요하다.

확인

현재 `pom.xml`에서 JDBC 의존성은 실제 DB 연동 전까지 주석 처리되어 있다. 그래서 지금 프로젝트는 DB 없이 `GET /` 요청을 받아 `templates/index.html`을 보여주는 Spring MVC + Thymeleaf 기본 구조로 이해하면 된다.

참고한 공식 문서:

- Spring Boot `@SpringBootApplication`:
<https://docs.spring.io/spring-boot/reference/using/using-the-springbootapplication-annotation.html>
- Spring Framework `DispatcherServlet`:
<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html>
- Spring Boot Servlet Web Applications / MVC / Template Engines:
<https://docs.spring.io/spring-boot/reference/web/servlet.html>

프로젝트 구조

```
D:/Study/Spring/SpringSample
├── pom.xml
├── src/main/java/com/example/springsample
│   ├── SpringSampleApplication.java
│   └── MainController.java
├── src/main/resources
│   ├── application.yaml
│   └── templates/index.html
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`pom.xml`은 Maven 프로젝트 설정 파일이고, `src/main/java`에는 Java 코드, `src/main/resources/templates`에는 Thymeleaf HTML 템플릿이 들어간다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

pom.xml 역할

`pom.xml`은 이 프로젝트가 어떤 Java 버전을 쓰고, 어떤 라이브러리를 가져오고, 어떻게 빌드할지를 정의한다.

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>4.1.0</version>
<relativePath/>
</parent>

<properties>
<java.version>21</java.version>
</properties>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`spring-boot-starter-parent`는 Spring Boot 프로젝트의 기본 Maven 설정을 제공한다. 의존성 버전 관리, 플러그인 기본값 같은 것을 Spring Boot 쪽에서 맞춰준다.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`spring-boot-starter-webmvc`는 Spring MVC 기반 웹 서버/컨트롤러 처리를 가능하게 하고, `spring-boot-starter-thymeleaf`는 HTML 템플릿 엔진인 Thymeleaf를 사용할 수 있게 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

DB 의존성 주석 처리 상태

```
<!-- 실제 DB연동할 때 활성화 할 예정임-->
<!-- <dependency>-->
<!-- <groupId>org.springframework.boot</groupId>-->
<!-- <artifactId>spring-boot-starter-jdbc</artifactId>-->
<!-- </dependency>-->
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`spring-boot-starter-jdbc`는 JDBC 방식으로 DB에 접근할 때 필요한 핵심 starter다. 지금은 DB 연결이 당장 어려워 주석 처리되어 있으므로, Spring Boot가 JDBC 관련 Bean을 적극적으로 구성하지 않는다.

```
<dependency>
<groupId>org.mariadb.jdbc</groupId>
<artifactId>mariadb-java-client</artifactId>
<scope>runtime</scope>
</dependency>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

MariaDB 드라이버는 남아 있지만, JDBC starter와 DB 접속 설정이 없으면 실제 DB 작업을 바로 하기는 어렵다.

```
spring:
  application:
    name: SpringSample
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

현재 `application.yml`에는 애플리케이션 이름만 있다. DB를 연결하려면 나중에 `spring.datasource.url`, `spring.datasource.username`, `spring.datasource.password` 같은 설정이 필요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

실행 시작점: main 함수

```
package com.example.springsample;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringSampleApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringSampleApplication.class, args);
    }

}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Java 프로그램은 `main` 함수에서 시작한다. Spring Boot 프로젝트도 결국 Java 프로그램이기 때문에 `main`에서 `SpringApplication.run(...)`을 호출하면서 서버가 시작된다.

공식 문서 기준으로 `@SpringBootApplication`은 세 가지 역할을 합친 애노테이션이다.

<code>@SpringBootApplication</code>	이 클래스가 Spring Boot 설정 클래스임을 표시
<code>@EnableAutoConfiguration</code>	classpath에 있는 의존성을 보고 필요한 설정을 자동 구성
<code>@ComponentScan</code>	현재 패키지과 하위 패키지에서 <code>@Controller</code> , <code>@Service</code> , <code>@Component</code> 등을 찾아 Bean으로 등록

주석 현재 `SpringSampleApplication`이 `com.example.springsample` 패키지에 있으므로, 같은 패키지의 `MainController`도 component scan 대상이 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

Spring Boot가 실행될 때 생기는 일

```
main()
-> SpringApplication.run(...)
-> Spring Container 생성
-> @SpringBootApplication 처리
-> Component Scan
-> MainController Bean 등록
-> Web MVC 자동 설정
-> 내장 웹 서버 실행
-> 요청을 받을 준비 완료
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Spring Boot는 코드를 직접 new 해서 쓰기보다, Spring Container가 객체를 만들고 관리한다. 이렇게 Spring이 관리하는 객체를 Bean이라고 부른다.

Controller 코드

```
package com.example.springsample;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MainController {

    @GetMapping("/")
    public String index(){
        return "index"; // app.render('index')
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `@Controller`가 붙어 있으므로 Spring은 `MainController`를 요청 처리용 Bean으로 등록한다.

주석 `@GetMapping("/")`는 HTTP GET 요청 중 경로가 `/`인 요청을 `index()` 메서드에 연결한다.

주석 `return "index";`는 문자열 자체를 응답하는 것이 아니라, 기본적으로 `index`라는 view 이름을 반환한다. Thymeleaf가 켜져 있으므로 Spring은 `src/main/resources/templates/index.html`을 찾아 렌더링한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<title>Title</title>
</head>
<body>
<h1>개쩌는 html</h1>
</body>
</html>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`templates/index.html`은 Controller가 `return "index";` 했을 때 찾아지는 HTML 파일이다. Spring Boot 공식 문서도 Thymeleaf 같은 template engine을 기본 설정으로 사용할 때 `src/main/resources/templates`에서 template을 자동으로 찾는다고 설명한다.

요청 처리 흐름

```
브라우저
-> GET /
-> 내장 웹 서버
-> DispatcherServlet
-> HandlerMapping
-> MainController.index()
-> return "index"
-> ViewResolver
-> templates/index.html
-> HTML 응답
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

Spring MVC는 `DispatcherServlet`이라는 중심 Servlet을 앞에 세운다. 공식 문서에서는 이 구조를 front controller pattern이라고 설명한다. 요청은 일단 `DispatcherServlet`으로 들어오고, 실제 처리는 Controller 같은 구성 요소가 담당한다.

Express와 비교해서 이해

이전에 했던 Express 코드와 비교하면 감이 더 쉽다.

```
app.get('/', function(req, res){
  res.render('index');
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Spring에서는 비슷한 흐름이 이렇게 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

```
@Controller
public class MainController {

    @GetMapping("/")
    public String index(){
        return "index";
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

비교:

Express `app.get('/')`	Spring `@GetMapping("/")`
---------------------------	---------------------------

- Express `res.render('index')`: Spring `return "index"``

Express `views/index.ejs`	Spring `templates/index.html`
Express middlewa re/router	Spring `DispatcherServlet + HandlerMapping + Controller`

주의 Express는 개발자가 `app.get`, `app.use` 순서를 직접 많이 관리하는 느낌이 강하고, Spring은 애노테이션을 보고 Spring Container와 MVC 자동 설정이 연결해 주는 느낌이 강하다.

DispatcherServlet이 하는 일

`DispatcherServlet`은 Spring MVC의 입구다.

```
요청이 들어옴
-> 어떤 Controller가 처리할지 찾음
-> 해당 메서드 실행
-> 반환값을 View 또는 Response로 해석
-> 최종 응답 생성
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `@GetMapping("/")` 같은 매핑 정보를 보고 어떤 메서드를 실행할지 찾는 과정이 있다. 그래서 Controller 메서드 이름이 `index`인지 아닌지보다, 애노테이션의 URL 매핑이 더 중요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

ViewResolver와 Thymeleaf

Controller가 `return "index";`를 하면 Spring은 이 문자열을 view 이름으로 본다.

```
return "index"
-> ViewResolver
-> templates/index.html
-> Thymeleaf 렌더링
-> HTML 응답
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 현재 HTML에는 동적 데이터가 없지만, 나중에는 Model에 데이터를 담아서 Thymeleaf에서 출력할 수 있다.

```
@GetMapping("/")
public String index(Model model){
    model.addAttribute("message", "hello");
    return "index";
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
<h1 th:text="${message}">기본 문구</h1>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `Model`은 Controller에서 View로 데이터를 넘기는 바구니 역할을 한다. Express의 `res.render('index', { message: 'hello' })`와 비슷하게 이해하면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

DB 연결을 나중에 켤 때 필요한 흐름

현재는 DB 연결을 일부러 빼 둔 상태다. 나중에 DB를 연결하려면 대략 아래 흐름이 필요하다.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

```
spring:
datasource:
url: jdbc:mariadb://192.168.100.230:3306/my_community
username: dietpi
password: 1234
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

비밀번호를 수업 실습용으로 직접 적을 수는 있지만, 실제 운영에서는 환경변수나 별도 secret 관리가 필요하다.

```
@Controller
public class MainController {
private final JdbcTemplate jdbcTemplate;

public MainController(JdbcTemplate jdbcTemplate) {
this.jdbcTemplate = jdbcTemplate;
}
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

JDBC starter와 datasource 설정이 준비되면 Spring Boot가 `DataSource`, `JdbcTemplate` 같은 DB 관련 Bean을 자동 구성할 수 있다. 지금은 이 단계를 일부러 미뤄 둔 상태다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 - Spring Boot 개념정리	date

지금 프로젝트의 전체 구동 요약

1. Maven이 pom.xml을 보고 의존성을 준비한다.
2. Java main 함수가 실행된다.
3. SpringApplication.run(...)이 Spring Boot를 시작한다.
4. @SpringBootApplication이 자동 설정과 component scan을 켜다.
5. MainController가 Bean으로 등록된다.
6. Spring MVC가 내장 웹 서버와 DispatcherServlet을 준비한다.
7. 브라우저가 GET / 요청을 보낸다.
8. DispatcherServlet이 @GetMapping("/")를 찾는다.
9. MainController.index()가 실행된다.
10. return "index"를 View 이름으로 해석한다.
11. templates/index.html을 찾아 HTML로 응답한다.

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

핵심 암기

<code>`pom.xml`</code>	의존성/빌드 설정
<code>`@SpringBootApplication`</code>	자동 설정 + component scan + 설정 클래스
<code>`SpringApplication.run`</code>	Spring Boot 앱 실행 시작
<code>`@Controller`</code>	요청을 처리하는 클래스
<code>`@GetMapping("/")`</code>	GET `/` 요청과 메서드 연결
<code>`return "index"`</code>	<code>`templates/index.html`</code> 을 렌더링하라는 의미
<code>`DispatcherServlet`</code>	Spring MVC 요청 처리의 중앙 입구
<code>`templates`</code>	Thymeleaf HTML 파일 위치
<code>`application.yml`</code>	앱 이름, 포트, DB 접속 정보 같은 설정 파일

정리

지금 SpringSample은 “DB 없는 첫 Spring MVC 화면 출력 프로젝트”다. 먼저 main → Spring Container → Controller 등록 → GET 요청 → ViewResolver → index.html 응답 흐름을 이해하고, DB는 나중에 JDBC starter와 datasource 설정을 켜 뒤 연결하면 된다.